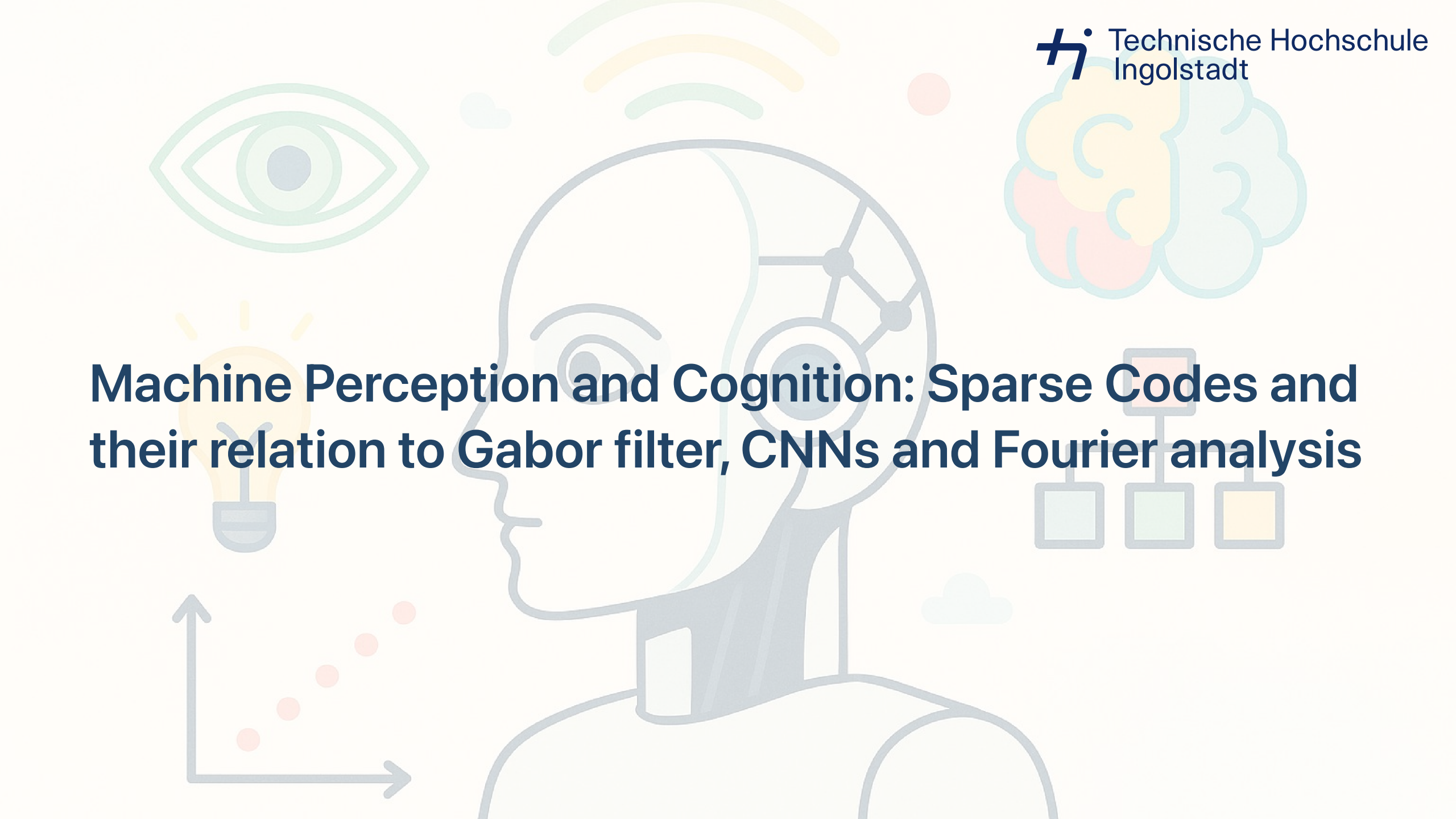
Technische Hochschule Ingolstadt



Machine Perception and Cognition: Sparse Codes and their relation to Gabor filter, CNNs and Fourier analysis

Learning Goals

After this lecture you will be able to:

- Formulate the **optimization objective** of sparse coding and explain the role of the dictionary $\mathbf{D}$ and the coefficient vector $\mathbf{c}$
- Describe how dictionary atoms emerge from natural images and why they resemble **Gabor filters**
- Explain the connection between sparse coding and the **Fourier transform** as competing linear decompositions
- Relate the sparse-coding framework to the computations inside a **CNN**: filters as dictionary atoms, Batch Normalization + ReLU as the sparsification mechanism that produces sparse activations (≈ sparse codes c_j), and how those sparse codes feed hierarchically into next-layer filters acting as new dictionary atoms for the next level

Motivation — Why Represent Images Efficiently?

Raw pixel arrays are high-dimensional and contain massive redundancy:

- A 32×32 grayscale patch has 1024 dimensions — but lives on a low-dimensional manifold
- Neighboring pixels are highly correlated
- Most of the "information" is encoded in a handful of local oriented structures

Goal: find a compact description $\mathbf{c}$ of an image (or patch) $\mathbf{x}$ such that

- $\mathbf{x}$ can be **faithfully reconstructed** from $\mathbf{c}$
- $\mathbf{c}$ is as **short** (sparse) as possible

The Dictionary Model

Key idea: represent an image patch $\mathbf{x} \in \mathbb{R}^d$ as a **linear combination of atoms** drawn from a dictionary $\mathbf{D} \in \mathbb{R}^{d \times k}$:

$$\mathbf{x} \approx \mathbf{D}\mathbf{c} = \sum_{j=1}^{k} c_j \, \mathbf{d}_j$$

- $\mathbf{d}_j \in \mathbb{R}^d$ — the j-th dictionary atom (a learned basis image), shared globally across all patches
- c_j — scalar coefficient for atom j, **local** to this patch; most entries are zero
- $k > d$ — overcomplete (more atoms than pixel dimensions): many redundant atoms allow sparser codes

Notation Overview

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^d$	one image patch ($d = \text{patch_size}^2$ pixels)
$\mathbf{D} \in \mathbb{R}^{d \times k}$	dictionary of k atoms, **learned globally**
$\mathbf{c} \in \mathbb{R}^k$	sparse coefficient vector, **local** to patch $\mathbf{x}$
k	number of dictionary atoms (e.g. 64)
d	number of pixels per patch (e.g. $8 \times 8 = 64$)

In `example_sparse_codes.py`: `N_ATOMS = 64`, `PATCH_SIZE = 8` $\Rightarrow k = d = 64$ (exactly complete), `N_NONZERO = 10` $\Rightarrow \|\mathbf{c}\|_0 \leq 10$.

The Sparse Coding Objective

We want to find both $\mathbf{D}$ and $\mathbf{c}$ by minimizing:

$$\min_{\mathbf{D},\,\mathbf{c}} \quad \underbrace{\|\mathbf{x} - \mathbf{D}\mathbf{c}\|_2^2}_{\text{reconstruction fidelity}} \qquad \text{subject to} \quad \|\mathbf{c}\|_0 \leq s$$

- **First term** — mean squared error (MSE): reconstruction should closely match the original patch
- **Constraint** — at most s non-zero coefficients: the code must be sparse

In practice the hard ℓ_0 constraint is often relaxed to an ℓ_1 penalty:

$$\min_{\mathbf{D},\,\mathbf{c}} \|\mathbf{x} - \mathbf{D}\mathbf{c}\|_2^2 + \lambda\|\mathbf{c}\|_1$$

Reconstruction Fidelity — MSE Term

$$\|\mathbf{x} - \mathbf{D}\mathbf{c}\|_2^2 = \sum_{i=1}^{d} \left(x_i - [\mathbf{D}\mathbf{c}]_i\right)^2$$

- Measures **pixel-wise squared error** between the original patch $\mathbf{x}$ and the reconstruction $\mathbf{D}\mathbf{c}$
- Drives the dictionary atoms to capture the actual structure of the patch
- Alone (without sparsity) this is just least-squares regression — perfectly solvable with $k \geq d$ atoms, but the solution is not sparse

The sparsity constraint forces the model to select only the **most relevant atoms** for each patch.

Sparsity — Why Does It Matter?

ℓ_0 **"norm"** — counts the number of non-zero entries:

$$\|\mathbf{c}\|_0 = \#\{i \mid c_i \neq 0\}$$

Minimizing $\|\mathbf{c}\|_0$ is NP-hard in general (combinatorial search).

ℓ_1 **norm** — convex surrogate:

$$\|\mathbf{c}\|_1 = \sum_i |c_i|$$

The ℓ_1 penalty shrinks small coefficients to exactly zero → sparse solution without combinatorial search.

Greedy alternative — Orthogonal Matching Pursuit (OMP): greedily select atoms one by one.

Dictionary Learning — Joint Optimization

Both $\mathbf{D}$ and $\mathbf{c}$ are unknown. The joint problem is non-convex, but convex in each variable separately:

Alternating optimization:

1. **Sparse encoding step** (fix $\mathbf{D}$, optimize $\mathbf{c}$):

$$\mathbf{c}^* = \arg\min_{\mathbf{c}} \|\mathbf{x} - \mathbf{D}\mathbf{c}\|_2^2 + \lambda\|\mathbf{c}\|_1$$

2. **Dictionary update step** (fix $\mathbf{c}$, optimize $\mathbf{D}$):

$$\mathbf{D}^* = \arg\min_{\mathbf{D}} \sum_n \|\mathbf{x}_n - \mathbf{D}\mathbf{c}_n\|_2^2 \quad \text{s.t.} \quad \|\mathbf{d}_j\|_2 = 1$$

The norm constraint on atoms prevents trivial scaling solutions.

Mini-Batch Dictionary Learning

1. Sample a mini-batch of patches
2. Encode each patch with OMP (sparse encoding)
3. Update dictionary atoms with a gradient step
4. Repeat over all mini-batches

Mini-Batch Dictionary Learning

In `example_sparse_codes.py` (scikit-learn):

```
learner = MiniBatchDictionaryLearning(
    n_components=64,           # number of atoms
    transform_algorithm="omp",
    transform_n_nonzero_coefs=10,  # sparsity level
)
learner.fit(whitened_patches)   # alternating OMP + dict. update
dictionary = learner.components_  # shape: (64, 64)
```

Orthogonal Matching Pursuit (OMP)

OMP is a greedy algorithm for sparse encoding. Given a fixed $\mathbf{D}$, find at most s active atoms for patch $\mathbf{x}$:

1. Initialize residual $\mathbf{r} = \mathbf{x}$, active set $\Lambda = \emptyset$
2. **Select**: $j^* = \arg\max_j \left|\langle \mathbf{d}_j, \mathbf{r} \rangle\right|$ — the atom most correlated with the residual
3. **Update**: $\Lambda \leftarrow \Lambda \cup \{j^*\}$, solve $\min_{\mathbf{c}} \left\|\mathbf{x} - \mathbf{D}_\Lambda \mathbf{c}\right\|_2^2$ (least-squares on selected atoms)
4. **Subtract**: $\mathbf{r} \leftarrow \mathbf{x} - \mathbf{D}_\Lambda \mathbf{c}$
5. Repeat from step 2 until $\left|\Lambda\right| = s$

Result: a coefficient vector with at most s non-zero entries.

Code Walk-Through — Data Pipeline

```
# 1. Load 500 CIFAR-10 images as float32 grayscale
images_gray = rgb_to_gray(load_cifar10_as_numpy(500))  # (500, 32, 32)

# 2. Extract 50 000 random 8×8 patches → flatten to vectors
patches = extract_random_patches(images_gray, PATCH_SIZE=8,
                                 n_patches=50000)  # (50000, 64)

# 3. ZCA whitening: remove mean, equalize variance across directions
whitened, mean, W, W_inv = whiten_patches(patches)

# 4. Learn dictionary: 64 atoms via mini-batch alternating optimization
dictionary = learn_sparse_dictionary(whitened, n_atoms=64)
# dictionary.shape == (64, 64)  — 64 atoms of size 64 pixels each
```

After training, `dictionary[i]` is a learned atom reshaped to 8×8 pixels.

ZCA Whitening — Why It Is Needed

Natural image patches have a **highly non-uniform power spectrum**: low frequencies (smooth regions) dominate and carry much more variance than high frequencies (fine edges and textures).

This imbalance is a problem for dictionary learning: without any correction, the algorithm spends most of its atoms on the most energetic (low-frequency) directions and largely ignores fine detail.

ZCA Whitening — Why It Is Needed

ZCA whitening is a preprocessing step that removes this imbalance — it rescales the data so that **every direction in patch space contributes equally**, regardless of how dominant it was before. The result looks visually similar to white noise with no particular dominant frequency — hence the name *whitening*.

Practically, after whitening the dictionary learning is free to allocate atoms uniformly across all orientations and frequency bands, producing the diverse Gabor-like atom set we observe.

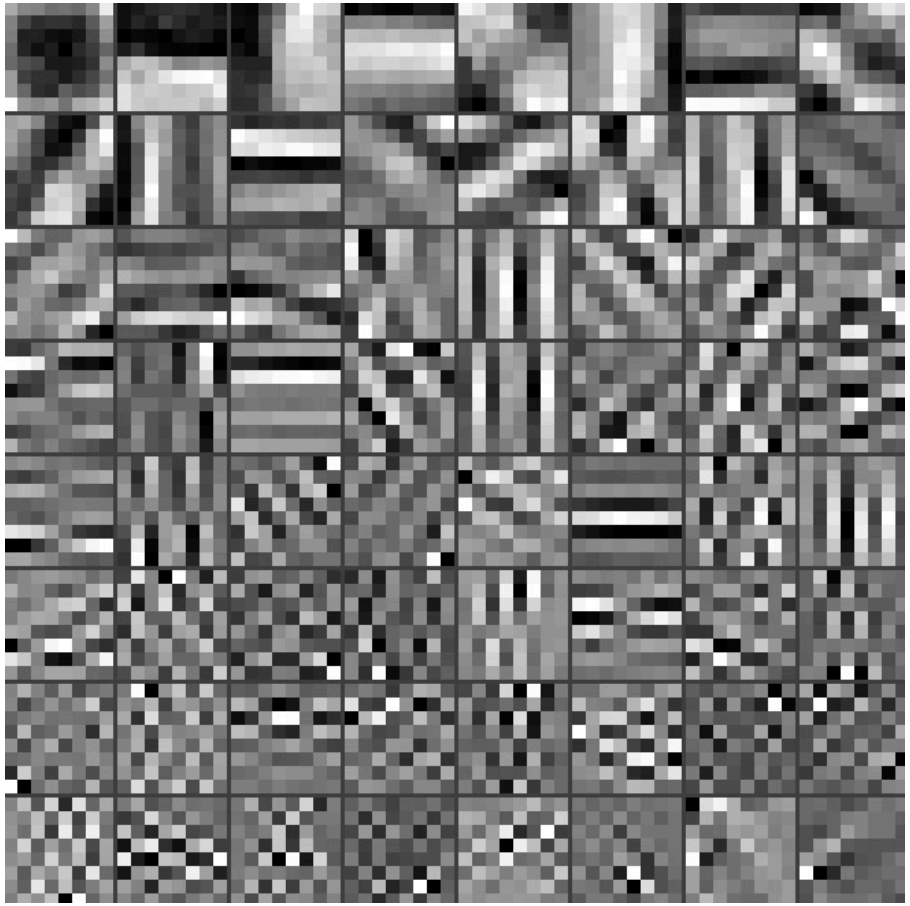
Learned Dictionary Atoms

After training on 50,000 natural image patches, the learned atoms in `dictionary` look like **oriented, localized, bandpass patterns** — they pick out edges and gratings at different orientations and spatial frequencies.

Each atom $\mathbf{d}_j$ captures a specific **orientation**, **spatial frequency**, and **phase** within the 8×8 patch. This is strikingly similar to the properties of Gabor filters — which we will now discuss.

Learned Dictionary Atoms

Output of `example_sparse_codes.py` based on CIFAR-10:



Code Walk-Through — Encoding and Reconstruction

```
# Encode one image patch-by-patch with OMP (at most 10 non-zero per patch)
codes, n_h, n_w = sparse_encode_image(test_image, dictionary,
                                      patch_mean, W_white,
                                      PATCH_SIZE, N_NONZERO=10)
# codes.shape == (n_patches, 64);  most entries are zero

# Reconstruct: linear combination of atoms + undo whitening
reconstructed = reconstruct_from_codes(codes, dictionary,
                                       patch_mean, W_inv,
                                       PATCH_SIZE, n_h, n_w)
```

Reconstruction (per patch i): the sparse coefficients $\mathbf{c}_i$ are combined with the dictionary atoms to obtain a whitened patch estimate; the whitening is then reversed and the mean is added back to recover the final pixel values.

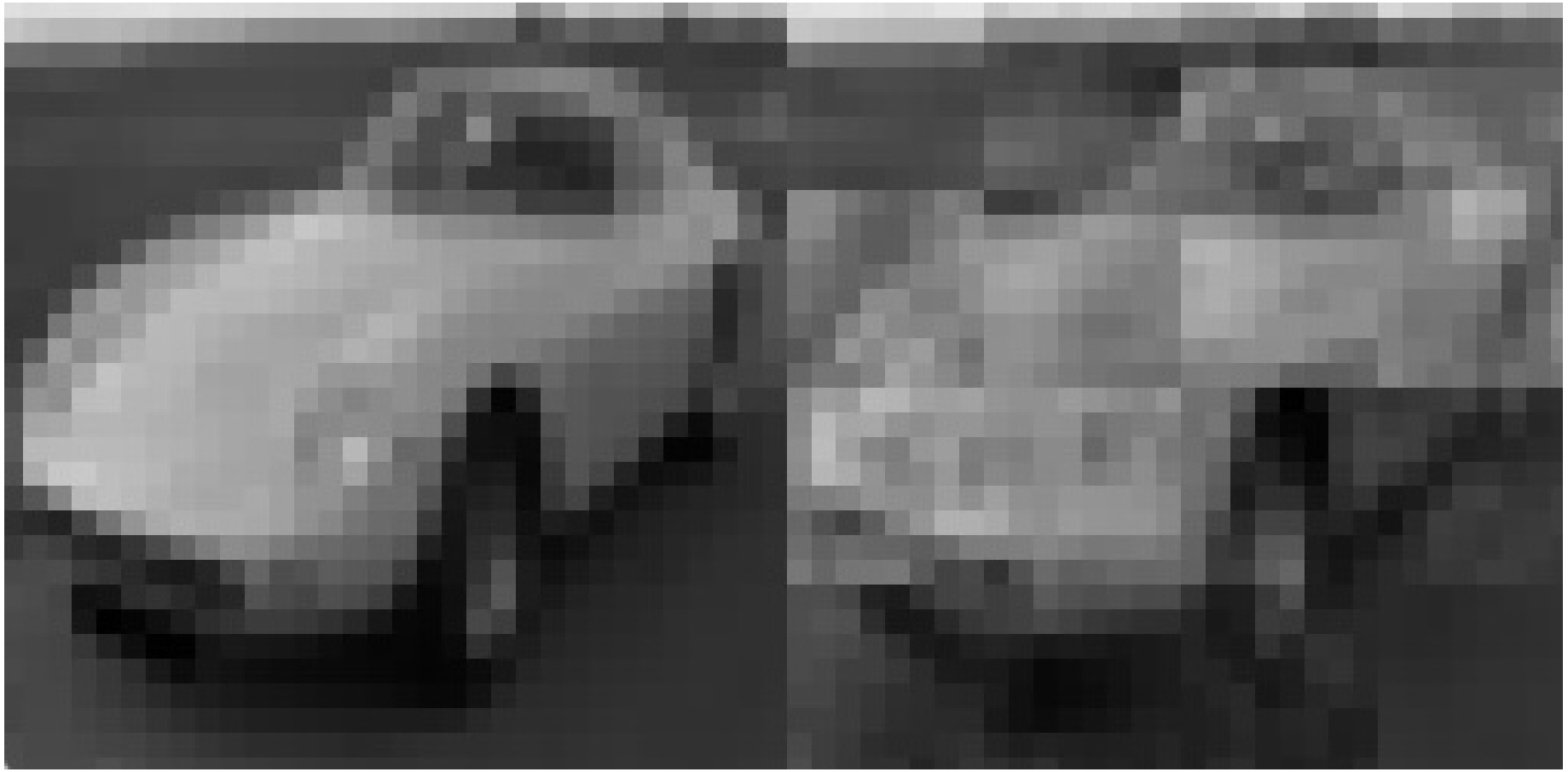
Reconstruction Quality

- Only 10 out of 64 atoms are active per 8×8 patch — **15.6 % non-zero**
- Despite this extreme compression, the reconstruction preserves edges, textures, and overall structure
- The mean absolute pixel error (reported by the script) is typically below 0.05 on CIFAR-10

This demonstrates that **oriented edge atoms efficiently capture the structure of natural images**.

Reconstruction Quality

Output of `example_sparse_codes.py` based on CIFAR-10:



Recap — What Is a Gabor Filter?

A 2-D **Gabor filter** is a sinusoidal grating windowed by a Gaussian:

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cos\left(\frac{2\pi x'}{\lambda} + \psi\right)$$

where $x' = x\cos\theta + y\sin\theta$ and $y' = -x\sin\theta + y\cos\theta$.

Key parameters:

- λ: wavelength of the sinusoid (spatial frequency)
- ψ: phase offset of the sinusoid
- θ: orientation of the filter
- σ: size of the Gaussian envelope (spatial extent)
- γ: spatial aspect ratio (ellipticity)

Gabor Filter Properties

Gabor filters are **bandpass** in both space and frequency:

- **Spatially localized**: the Gaussian envelope limits the filter's extent — only a local region of the image contributes to the response
- **Frequency selective**: the sinusoidal carrier selects a narrow band of spatial frequencies
- **Orientation selective**: response is maximal for a specific edge orientation

Biological Vision — Simple Cells in V1

Hubel and Wiesel (1962) discovered **simple cells** in the primary visual cortex (V1) of cats that respond selectively to oriented edges in specific retinal positions.

Daugman (1985) showed that 2-D Gabor functions are **theoretically optimal** for joint localization in space, spatial frequency, and orientation and argued that V1 simple-cell filters approximate these optimal functions. Jones and Palmer (1987) subsequently provided systematic empirical evidence that 2-D Gabor functions give a good quantitative fit to measured simple-cell receptive fields.

Biological Vision — Simple Cells in V1

Conservative interpretation for this course:

> The first stage of the visual hierarchy in mammals appears to perform something analogous to local oriented bandpass filtering, and Gabor functions are a good mathematical approximation of those filters.

This course is not about neuroscience! 😄

Olshausen & Field 1996 — The Key Insight

Olshausen and Field (1996) asked: if we seek a dictionary that allows natural image patches to be represented with as few active atoms as possible, what shape do the learned atoms have?

Answer: atoms that are **localized**, **oriented**, and **bandpass** — qualitatively matching the receptive fields of V1 simple cells.

Significance: the Gabor-like structure of V1 cells is not hand-engineered — it is the *optimal sparse code* for the statistics of natural images.

Why Do Sparse Atoms Look Like Gabor Filters?

Natural images have a specific statistical structure:

1. **Adjacent pixels are correlated**
2. **Natural scenes contain oriented contours**
3. **Low frequencies dominate in natural images**

An atom that captures a **local oriented edge** can explain many image patches with a single large coefficient, while atoms without orientation selectivity need many coefficients.

The sparsity objective therefore pushes atoms toward the shapes that are most informative about natural image structure — and those shapes are Gabor-like.

Dictionary Atoms vs. Gabor Filters — Side by Side

Property	Gabor filter	Learned sparse-code atom
Shape	Sinusoid × Gaussian	Sinusoid-like × localized
Orientation	Freely chosen	Emerges from data
Spatial frequency	Freely chosen	Emerges from data
Spatial localization	Yes (Gaussian window)	Yes (patch-sized)
Phase	Selectable	Emerges from data

The learned atoms are **not identical** to Gabor filters (they have data-driven irregularities) but share all key qualitative properties.

Recap — The Fourier Transform

Any signal $x(t)$ can be expressed as a sum of sinusoids (animated):



Image source: Lucas Vieira, Public domain, via Wikimedia Commons from https://commons.wikimedia.org/wiki/File:Fourier_transform_time_and_frequency_domains.gif

Fourier Transform as a Linear Decomposition

The Fourier transform is a **linear decomposition** of the image into a fixed set of basis functions:

$$\mathbf{x} = \mathbf{F}^{-1}\hat{\mathbf{x}}$$

- $\mathbf{F}$ — Fourier transform matrix (fixed, not learned)
- $\hat{\mathbf{x}}$ — Fourier coefficients (the "code" for this image)
- Reconstruction is a **linear combination** of sinusoidal basis functions weighted by the coefficients

This is structurally identical to the sparse coding model $\mathbf{x} \approx \mathbf{D}\mathbf{c}$ — the key differences lie in the choice of basis and the sparsity of the coefficients.

Fourier Transform vs. Sparse Coding

Property	Fourier transform	Sparse coding
Basis functions	Fixed global sinusoids	Learned local atoms
Coefficients	Generally all non-zero (dense)	Most are zero (sparse)
Locality	Global — each coefficient uses the whole image	Local — each atom covers a small patch
Adaptability	Fixed for all images	Dictionary learned from data
Reconstruction	Exact	Approximate (with sparsity constraint)
Computation	$O(N \log N)$ via FFT	Iterative

From Fourier to Sparse Codes — The Progression

$$\underbrace{\text{Fourier basis}}_{\text{fixed, global, dense}} \rightarrow \underbrace{\text{Gabor filters}}_{\text{fixed, local, bandpass}} \rightarrow \underbrace{\text{Sparse codes}}_{\text{learned, local, sparse}}$$

Each step improves the match to natural image statistics:

1. **Fourier**: global sinusoids — good for stationary signals, not images
2. **Gabor**: windowed sinusoids — local and oriented, but fixed and manually chosen
3. **Sparse codes**: learned local atoms — adapt to the actual statistics of the data

All three are **linear decompositions**: image = dictionary $\times$ coefficients.

Convolution as Template Matching

A 2D convolution of an input feature map $\mathbf{x}$ with a filter $\mathbf{w}$ at position (r, c):

$$(\mathbf{x} * \mathbf{w})[r, c] = \sum_{u,v} x[r+u,\ c+v]\, w[u, v]$$

This computes the **inner product** between the filter and the local image patch centered at (r, c).

The output map contains one value per position: how strongly the filter's pattern appears at that location. This is exactly the same operation as **projecting a patch onto a dictionary atom** in sparse coding.

CNN Filters as Dictionary Atoms

In a convolutional layer with k filters $\{\mathbf{w}_1, \ldots, \mathbf{w}_k\}$:

$$a_j[r, c] = \langle \mathbf{x}_{r,c},\ \mathbf{w}_j \rangle \quad \text{for } j = 1, \ldots, k$$

- $\mathbf{x}_{r,c}$ — local patch of the input at position (r, c)
- $a_j[r, c]$ — **raw projection** of the local patch onto filter j at position (r, c)
- The full set of raw projections $\{a_j[r, c]\}_{j,r,c}$ is the **feature map**

Analogy: each CNN filter $\mathbf{w}_j$ is the **dictionary atom** $\mathbf{d}_j$ — the convolution computes the inner product $\langle \mathbf{x}_{r,c}, \mathbf{w}_j \rangle$, exactly as sparse coding projects a patch onto an atom to measure similarity. The raw projections $a_j[r, c]$ are **dense** (all filters respond to every patch) and are therefore not yet the sparse coefficients; Batch Normalization and ReLU (next slides) enforce sparsity and produce the actual sparse activations $\hat{a}_j[r, c] \approx c_j$.

First-Layer CNN Filters ≈ Gabor Filters

As confirmed by visualization (Zeiler & Fergus, 2014) and the previous lecture:

- CNN filters in the **first layer**, trained on natural images by gradient descent, converge to **Gabor-like oriented edge detectors**
- This matches exactly what sparse dictionary learning produces for the same data
- Both CNNs and sparse coding discover that oriented, localized, bandpass features are the most useful atoms for natural images

This is not a coincidence: **both are optimizing a representation for a large corpus of natural images**, and the statistics of those images favor Gabor-like atoms.

ReLU as a Sparsity Prior

After computing activations, a CNN applies **Rectified Linear Unit (ReLU)**:

$$\sigma(a) = \max(0, a)$$

- Sets **negative activations to zero** — filter responses that do not match the pattern are suppressed
- Only a subset of filters "fire" at any given position → **sparse activation map**
- This is an analogy of the **sparsity constraint** $\|\mathbf{c}\|_0 \leq s$ in sparse coding

ReLU does not select the top-k atoms explicitly (unlike OMP), but it implicitly enforces a population sparsity: most units are inactive for any given input. Together with Batch Normalization (see next slide), ReLU converts the dense raw projections $a_j[r, c]$ into sparse activations $\hat{a}_j[r, c]$ — the CNN's counterpart of the sparse coefficients c_j.

Batch Normalization — Normalizing the Codes

Batch Normalization transforms activations within a mini-batch in two steps:

$$\hat{a}_j = \frac{a_j - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}, \qquad y_j = \gamma_j \, \hat{a}_j + \beta_j$$

- μ_j, σ_j^2: batch mean and variance for feature j; ε: small constant for numerical stability
- γ_j, β_j: learnable scale and shift parameters that restore the layer's representational capacity
- The normalization step removes the mean and equalizes variance across the batch
- Prevents individual filters from dominating
- Combined with ReLU: activations are centered and only positive responses survive

Batch Normalization — Normalizing the Codes

These post-BN + ReLU values play a role analogous to **sparse codes** — the CNN's counterpart of the sparse coefficients c_j in the sparse coding model. Just as the sparse codes c_j serve as synthesis weights in $\hat{\mathbf{x}} = \sum_j c_j \mathbf{d}_j$ (weighting the atoms to produce a reconstruction), the sparse activations $\hat{a}_j$ serve as inputs to the next layer's convolutions. The next-layer filters $\mathbf{w}_{j,f}^{(l+1)}$ then act as **new dictionary atoms** for the next hierarchical level, combining the sparse activations to build a higher-level representation.

From Projections to Sparse Codes — The Three-Step Pipeline

The CNN computes a sparse representation of each local patch in three explicit steps:

Step	Sparse coding	CNN
1. Project onto atoms	$a_j = \langle \mathbf{x}, \mathbf{d}_j \rangle$ (dense, all atoms respond)	Convolution: $a_j[r, c] = \langle \mathbf{x}_{r,c}, \mathbf{w}_j \rangle$ (dense feature map)
2. Sparsify	OMP / ℓ_1 — keep only s coefficients	Batch Normalization + ReLU — zero out weak responses
3. Use sparse codes	$\hat{\mathbf{x}} = \sum_j c_j \mathbf{d}_j$ — sparse codes c_j weight atoms $\mathbf{d}_j$ to reconstruct $\mathbf{x}$	Next-layer filters (new atoms $\mathbf{w}_{j,f}^{(l+1)}$) convolve with sparse activations $\hat{a}_j^{(l)}$ ($\approx c_j$): $\sum_j \mathbf{w}_{j,f}^{(l+1)} * \hat{a}_j^{(l)}$

From Projections to Sparse Codes — The Three-Step Pipeline

- Steps 1–2 produce the **sparse code**: in sparse coding the sparse $\mathbf{c}$; in CNN the post-BN+ReLU activation maps $\hat{a}_j$
- Step 3 shows **how the sparse codes are used**: in sparse coding, the sparse codes c_j are the synthesis weights — they combine with atoms $\mathbf{d}_j$ to reconstruct $\hat{\mathbf{x}} = \sum_j c_j \mathbf{d}_j$; analogously, the sparse activations $\hat{a}_j^{(l)}$ ($\approx c_j$) serve as inputs to the next layer, where the next-layer filters $\mathbf{w}_{j,f}^{(l+1)}$ act as **new dictionary atoms** for the next hierarchical level

Hierarchical Codes — Deeper Layers

In sparse coding, one dictionary describes patches at **one scale**. In a CNN, each layer builds representations from the outputs of the layer below via a **linear combination**:

$$\hat{a}_f^{(l+1)}[r,c] = \sigma\left(\sum_{j=1}^{C_{\text{in}}} \left(\mathbf{w}_{j,f}^{(l+1)} * \hat{a}_j^{(l)}\right)[r,c]\right)$$

- $\hat{a}_j^{(l)}$ — sparse activation map j from layer l (the sparse code that layer l produced, analogous to c_j)
- $\mathbf{w}_{j,f}^{(l+1)}$ — filter of layer $l+1$ connecting input map j to output map f (acting as a new dictionary atom for the next level)

Hierarchical Codes — Deeper Layers

Analogy: the sparse activations $\hat{a}_j^{(l)}$ ($\approx c_j$) play the role of sparse codes — just as c_j weight atoms $\mathbf{d}_j$ in the synthesis $\hat{\mathbf{x}} = \sum_j c_j \mathbf{d}_j$, the $\hat{a}_j^{(l)}$ feed into the next-layer filters $\mathbf{w}_{j,f}^{(l+1)}$, which act as new dictionary atoms for the next hierarchical level.

CNN vs. Sparse Coding — Key Comparison

Aspect	Sparse coding	CNN
Dictionary atoms	Learned $\mathbf{D}$	Convolutional filters $\mathbf{W}^{(l)}$
Raw projections	$\langle \mathbf{x}, \mathbf{d}_j \rangle$ (dense)	Pre-BN+ReLU activations $a_j[r, c]$ (dense)
Sparse codes (coefficients)	Sparse $\mathbf{c}$ via OMP / ℓ_1	Post-BN + ReLU activations $\hat{a}_j[r, c]$
How sparse codes are used	c_j weight atoms $\mathbf{d}_j$ in synthesis: $\hat{\mathbf{x}} = \sum_j c_j \mathbf{d}_j$ (reconstruction)	$\hat{a}_j^{(l)}$ ($\approx c_j$) feed next-layer convolutions; next-layer filters $\mathbf{w}_{j,f}^{(l+1)}$ act as new dictionary atoms
Depth	Single-layer	Multi-layer hierarchy

Why Is the CNN Analogy Useful?

Understanding the sparse-coding analogy helps explain **why CNNs work**:

1. **Filter sharing (convolution)** — the same atom is tested at every position, matching the local nature of dictionary atoms
2. **BN + ReLU sparsity** — BatchNorm normalizes and ReLU suppresses weak responses; together they convert dense projections into sparse activations
3. **Sparse activations as synthesis inputs** — the sparse activations $\hat{a}_j^{(l)}$ serve as inputs to the next layer's convolutions; the next-layer filters act as new dictionary atoms for the next hierarchical level

Why Is the CNN Analogy Useful?

Continuation:

4. **Hierarchical composition** — deeper dictionaries build complex representations from simpler ones
5. **Gabor convergence** — the first-layer filters converge to Gabor-like atoms because those are optimal for natural image statistics

Differences — Discriminative vs. Generative

Important distinction:

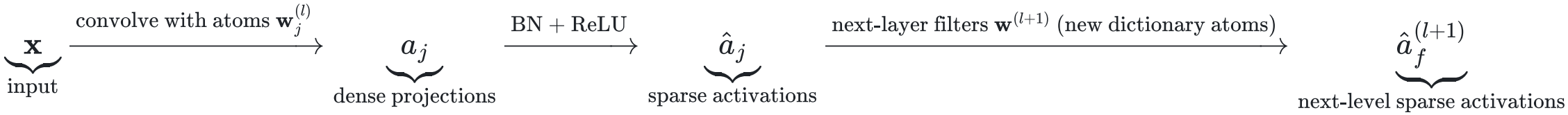
- **Sparse coding** is an **unsupervised generative model**: it optimizes reconstruction fidelity. The objective is: "can I reconstruct $\mathbf{x}$ from $\mathbf{Dc}$?"
- **CNNs** (used for classification) are **discriminative models**: they optimize a task-specific loss. The objective is: "can I predict the label y from $\mathbf{x}$?"

Differences — Discriminative vs. Generative

Consequently:

- CNN filters are shaped by **class boundaries**, not just image statistics
- The sparse-coding analogy is strongest in the **first layer** where no task pressure has accumulated yet
- Deeper CNN layers diverge from sparse coding: they discard information irrelevant to the task

The Unified View



Both sparse coding and CNNs implement a **project → sparsify → use/compose** pipeline:

- **Sparse coding**: project $\mathbf{x}$ onto atoms $\mathbf{D}$ → enforce sparsity (ℓ_1/OMP) → sparse codes $\mathbf{c}$ → synthesize $\hat{\mathbf{x}} = \mathbf{D}\mathbf{c}$ (sparse codes c_j weight atoms $\mathbf{d}_j$ to reconstruct)
- **CNN** (analogy): convolve with layer-l filters (≈ atoms) → BN + ReLU sparsification → sparse activations $\hat{a}_j$ (≈ c_j) → next-layer filters (≈ new dictionary atoms) produce next-level sparse codes (supervised, no explicit reconstruction)

Practical Exercise — Running the Code

In `example_sparse_codes.py`

What to observe:

1. The **learned dictionary atoms** window — do they look like oriented Gabor filters?
2. The **reconstruction windows** — how similar is the reconstruction to the original with only 10 active atoms out of 64?
3. The **console output** — inspect which atoms fire for the first three patches. Is the code truly sparse?

Experiment with `N_NONZERO`: decrease it to 3 (very sparse) or increase to 30. How does reconstruction quality change?

Experiment — Effect of Sparsity Level

`N_NONZERO`	Active atoms	Compression	Expected reconstruction
1	1 / 64 (1.6 %)	Very high	Only dominant edge visible
5	5 / 64 (7.8 %)	High	Main structure preserved
10	10 / 64 (15.6 %)	Moderate	Good reconstruction
30	30 / 64 (46.9 %)	Low	Near-perfect
64	64 / 64 (100 %)	None	Exact (least-squares)

The default `N_NONZERO = 10` is chosen to balance sparsity and quality.

The Big Picture — Three Perspectives on Images

All three representations share the same structure: **signal = linear combination of basis functions weighted by coefficients**.

Fourier — fixed global sinusoids, dense coefficients.
Sparse coding — learned local atoms, sparse coefficients.
CNN — learned filters, sparse activations (batch-normalization + ReLU).

Why Local and Oriented Atoms Win

Natural images are characterized by:

- **Statistical locality**: correlations decay rapidly with distance — a pixel tells us little about a pixel far away
- **Oriented structure**: object boundaries, shadows, and textures are locally directional
- **Scale invariance**: similar structures appear at multiple scales

The Olshausen & Field result is remarkable because it says: **if you ask for sparsity on natural images, you get Gabor-like atoms for free** — no manual design required.

Summary — Sparse Codes

- An image patch $\mathbf{x}$ is represented as $\mathbf{x} \approx \mathbf{D}\mathbf{c}$ where $\mathbf{D}$ is a **globally learned dictionary** and $\mathbf{c}$ is a **locally sparse coefficient vector**
- The optimization minimizes **MSE reconstruction error** subject to a **sparsity constraint** (ℓ_0 or ℓ_1)
- Dictionary learning alternates between **sparse encoding** (OMP) and **dictionary update** (gradient step)
- Learned atoms from natural images are **localized, oriented, and bandpass** — qualitatively matching Gabor filters

Summary — Relation to Gabor Filters

- Gabor filters are **spatially localized sinusoids** — theoretically optimal for joint localization in space, spatial frequency, and orientation (Daugman, 1985)
- Simple cells in the primary visual cortex (V1) of mammals have receptive fields well described by Gabor functions (Jones & Palmer, 1987)
- Olshausen & Field (1996): **sparsity on natural images ⟹ Gabor-like atoms** emerge without any manual design
- The shared structure (orientation, frequency selectivity, locality) arises from the statistical properties of natural images

Summary — Relation to Fourier Transform

- Both Fourier and sparse coding are **linear decompositions**: image = basis $\times$ coefficients
- Fourier: **fixed global basis** (sinusoids over the full image), generally **dense** coefficients
- Sparse coding: **learned local basis** (atoms on patches), explicitly **sparse** coefficients
- The key advantage of sparse codes: locality makes the representation efficient for natural images which are dominated by local structure

Summary — Relation to CNNs

- CNN convolutional filters $\mathbf{w}_j$ play the role of **dictionary atoms**: each filter tests for a specific local pattern at every position
- Convolution produces **dense raw projections** $a_j[r, c] = \langle \mathbf{x}_{r,c}, \mathbf{w}_j \rangle$ — these are analogous to projecting a patch onto atoms, not yet sparse coefficients
- **Batch Normalization + ReLU** enforce sparsity: BN normalizes and ReLU zeros out weak responses — the post-BN+ReLU values $\hat{a}_j$ are the **sparse activations**, i.e., the CNN's counterpart of the sparse coefficients c_j
- The sparse activations $\hat{a}_j$ (the CNN's sparse codes) play the role of the **synthesis coefficients** c_j — just as c_j weight atoms $\mathbf{d}_j$ in $\hat{\mathbf{x}} = \sum_j c_j \mathbf{d}_j$, the $\hat{a}_j^{(l)}$ feed into next-layer filters $\mathbf{w}_{j,f}^{(l+1)}$ which act as new dictionary atoms for the next hierarchical level
- **First-layer CNN filters** trained on natural images converge to **Gabor-like patterns**

Looking Ahead

The sparse-coding framework is foundational to understanding several upcoming topics:

- **Autoencoders**: an unsupervised encoder-decoder architecture that learns compressed representations — a non-linear generalization of sparse coding
- **Variational autoencoders**: impose a probabilistic prior on the latent code, generalizing the sparsity prior
- **Attention mechanisms and transformers**: dynamically select which "atoms" (tokens) are relevant for each query — a soft, data-driven version of OMP selection

The recurring theme: **efficient representations** are sparse, hierarchical, and adapted to the statistics of the data.

Final Recap

- 🧩 **Sparse codes**: $\mathbf{x} \approx \mathbf{D}\mathbf{c}$, minimize MSE + sparsity penalty; dictionary $\mathbf{D}$ is global, coefficients $\mathbf{c}$ are local and sparse
- 🌊 **Gabor connection**: learned atoms match Gabor functions; both capture local, oriented, bandpass structure; biologically motivated (V1 simple cells)
- 📐 **Fourier connection**: Fourier is a dense linear decomposition with fixed global sinusoids; sparse coding replaces global basis with local learned atoms for greater efficiency
- 🧠 **CNN connection**: filters ≈ dictionary atoms, convolution ≈ projection onto atoms (dense), BN + ReLU ≈ sparsification producing sparse activations $\hat{a}_j \approx c_j$ (the CNN's sparse codes), sparse activations ≈ synthesis inputs (c_j weight atoms in $\hat{\mathbf{x}} = \mathbf{D}\mathbf{c}$; analogously $\hat{a}_j^{(l)}$ feed next-layer filters acting as new dictionary atoms), depth ≈ hierarchical coding